

Параллельное программирование

## **Отчет**

**Лабораторная работа №6: Параллельная реализация решения уравнения теплопроводности**

Выполнил: группа 24940, МIRONЕНКО Павел

Дата: 04.06.2026

## Цель работы

Разработать и реализовать итерационный алгоритм решения стационарного уравнения теплопроводности (метод Якоби, пятиточечный разностный шаблон) для двумерной области. Расчеты проводятся на равномерных сетках размерами  $128 \times 128$ ,  $256 \times 256$ ,  $512 \times 512$  и  $1024 \times 1024$ . Граничные условия формируются посредством линейной интерполяции между угловыми точками, имеющими значения 10, 20, 30 и 20. Внутренние узлы сетки инициализируются нулями.

Необходимо достичь точности вычислений

$10^{-6}$  при лимите в  $10^6$  итераций. Также требуется портировать код на графический ускоритель с применением директив OpenACC, провести оптимизацию обменов памятью и сохранить итоговую матрицу температур для верификации корректности работы алгоритма.

## Используемый компилятор

Для компиляции кода использовался компилятор `nvc++` (входит в состав NVIDIA HPC SDK). Сборка осуществлялась с флагами `-acc` (для включения поддержки OpenACC) и `-Minfo=all` (для получения подробного отчета компилятора о распараллеливании циклов). В проекте также предусмотрен `CMakeLists.txt`, который позволяет запускать локальные тесты через стандартный `g++` без поддержки OpenACC — это использовалось для отладки логики, проверки граничных условий и корректности формата вывода.

## Используемый профилировщик

В качестве инструмента профилирования выбран NVIDIA Nsight Systems. Чтобы избежать раздувания размера трассировочного файла и упростить анализ таймлайна, профильные запуски проводились с искусственным ограничением числа итераций до 50–100.

Пример команды для снятия профиля:

```
nsys profile --stats=true -o trace_gpu_512 ./heat_solver -n 512 --eps 1e-6 --max-iter 100 -o out_profile.txt
```

## Как производили замер времени работы

Хронометраж выполнения вычислений встроен непосредственно в код программы с использованием стандартной библиотеки `<chrono>` (тип `high_resolution_clock`). Измеряется исключительно время итерационного процесса: засекается момент перед вызовом функции `solve(...)` и сразу после выхода из нее.

# Проверка (матрицы 10\*10 и 13\*13)

|               |               |               |               |               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| 10            | 11.1111111111 | 12.2222222222 | 13.3333333333 | 14.4444444444 | 15.5555555555 | 16.6666666667 | 17.7777777778 | 18.8888888889 | 20            |
| 11.1111111111 | 12.2222204449 | 13.3333300009 | 14.444399546  | 15.555504499  | 16.66661561   | 17.777732879  | 18.888855564  | 19.999982268  | 21.1111111111 |
| 12.2222222222 | 13.3333000009 | 14.444381814  | 15.5555471174 | 16.6666570712 | 17.7777681823 | 18.888804508  | 19.999993737  | 21.1111077786 | 22.2222222222 |
| 13.3333333333 | 14.4444399546 | 15.5555471174 | 16.666655298  | 17.7777648498 | 18.888759609  | 19.9999886313 | 21.111102673  | 22.2222177324 | 23.3333333333 |
| 14.4444444444 | 15.555504499  | 16.6666570712 | 17.7777648498 | 18.888741877  | 19.999852988  | 21.110981831  | 22.222126267  | 23.333282277  | 24.4444444444 |
| 15.5555555556 | 16.666661561  | 17.7777681823 | 18.888759609  | 19.999852988  | 21.11109641   | 22.2222092942 | 23.333237378  | 24.444393388  | 25.5555555556 |
| 16.6666666667 | 17.7777732879 | 18.888804508  | 19.9999886313 | 21.1110981831 | 22.2222092942 | 23.333219647  | 24.444360063  | 25.555510657  | 26.6666666667 |
| 17.7777777778 | 18.888855564  | 19.999993737  | 21.111102673  | 22.2222126267 | 23.333237378  | 24.444360063  | 25.5555492925 | 26.666633342  | 27.7777777778 |
| 18.8888888889 | 19.999982268  | 21.1111077786 | 22.2222177324 | 23.333282277  | 24.444393388  | 25.555510657  | 26.666633342  | 27.7777760046 | 28.8888888889 |
| 20            | 21.1111111111 | 22.2222222222 | 23.3333333333 | 24.4444444444 | 25.5555555556 | 26.6666666667 | 27.7777777778 | 28.8888888889 | 30            |

|               |               |               |               |               |               |               |               |               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| 10            | 10.8333333333 | 11.6666666667 | 12.5          | 13.3333333333 | 14.1666666667 | 15            | 15.8333333333 | 16.6666666667 | 17.5          | 18.3333333333 | 19.1666666667 | 20            |
| 10.8333333333 | 11.666647639  | 12.4999963263 | 13.3333281348 | 14.166660836  | 14.9999928987 | 15.8333250859 | 16.6666595654 | 17.499993637  | 18.3333281348 | 19.166662993  | 19.999980972  | 20.8333333333 |
| 11.6666666667 | 12.4999963263 | 13.333326232  | 14.1666566299 | 14.9999877082 | 15.8333196229 | 16.666652464  | 17.4999862896 | 18.3333210335 | 19.1666566299 | 19.9999928987 | 20.8333295596 | 21.6666666667 |
| 12.5          | 13.3333281348 | 14.1666566299 | 14.9999857974 | 15.8333159492 | 16.6666472655 | 17.4999799265 | 18.3333193222 | 19.1666492825 | 19.9999857974 | 20.833323966  | 21.6666614681 | 22.5          |
| 13.3333333333 | 14.166660836  | 14.9999877082 | 15.8333159492 | 16.6666453627 | 17.4999762528 | 18.3333087337 | 19.1666429195 | 19.999978961  | 20.8333159492 | 21.666643668  | 22.499993637  | 23.3333333333 |
| 14.1666666667 | 14.9999877082 | 15.8333159492 | 16.6666472655 | 17.4999762528 | 18.3333087337 | 19.1666392458 | 19.9999734975 | 20.8333085862 | 21.6666472655 | 22.4999862896 | 23.333326232  | 24.1666666667 |
| 15            | 15.8333250859 | 16.666652464  | 17.4999799265 | 18.3333087337 | 19.1666392458 | 19.9999715948 | 20.8333059125 | 21.6666429195 | 22.4999799265 | 23.333319322  | 24.1666593193 | 25            |
| 15.8333333333 | 16.6666595654 | 17.4999862896 | 18.3333193222 | 19.1666429195 | 19.9999734975 | 20.8333059125 | 21.6666401642 | 22.4999762528 | 23.3333193222 | 24.1666529562 | 24.9999928987 | 25.8333333333 |
| 16.6666666667 | 17.499993637  | 18.3333210335 | 19.1666566299 | 19.9999857974 | 20.8333159492 | 21.6666429195 | 22.4999762528 | 23.3333193222 | 24.1666492825 | 24.9999857974 | 25.833323966  | 26.6666666667 |
| 17.5          | 18.3333281348 | 19.1666566299 | 19.9999857974 | 20.8333159492 | 21.6666472655 | 22.4999799265 | 23.3333193222 | 24.1666492825 | 24.9999857974 | 25.833323966  | 26.6666614681 | 27.5          |
| 18.3333333333 | 19.166662993  | 19.9999928987 | 20.833323966  | 21.6666529562 | 22.4999862896 | 23.3333193222 | 24.1666529562 | 24.9999877082 | 25.833323966  | 26.6666595654 | 27.4999963263 | 28.3333333333 |
| 19.1666666667 | 19.9999928987 | 20.833326232  | 21.6666614681 | 22.499993637  | 23.333326232  | 24.1666593193 | 24.9999928987 | 25.833326232  | 26.6666614681 | 27.4999963263 | 28.333314385  | 29.1666666667 |
| 20            | 20.8333333333 | 21.6666666667 | 22.5          | 23.3333333333 | 24.1666666667 | 25            | 25.8333333333 | 26.6666666667 | 27.5          | 28.3333333333 | 29.1666666667 | 30            |

## Выполнение на CPU

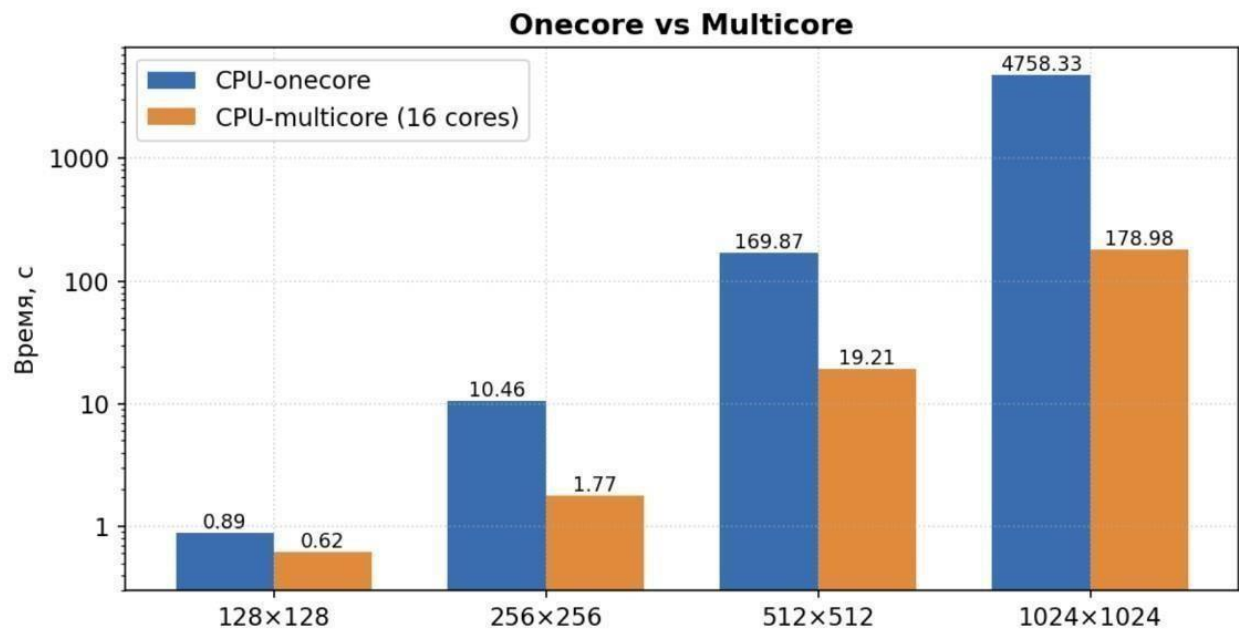
### CPU-onescore

| Размер сетки | Время выполнения | Точность     | Количество итераций |
|--------------|------------------|--------------|---------------------|
| 128*128      | 0.89 с           | 9.999803e-07 | 30074               |
| 256*256      | 10.46 с          | 9.999302e-07 | 102885              |
| 512*512      | 169.87 с         | 9.999302e-07 | 339599              |
| 1024*1024    | 4758.33 с        | 1.36929e-06  | 1000000             |

### CPU-multicore

| Размер сетки | Время выполнения | Точность     | Количество итераций |
|--------------|------------------|--------------|---------------------|
| 128*128      | 0.62 с           | 9.999302e-07 | 30074               |
| 256*256      | 1.77 с           | 9.999302e-07 | 102885              |
| 512*512      | 19.21 с          | 9.999302e-07 | 339599              |
| 1024*1024    | 178.98 с         | 1.36929e-06  | 1000000             |

Диаграмма сравнения время работы CPU-one и CPU-multi



Выполнение на GPU

Этапы оптимизации на сетке 512\*512

| Этап № | Время выполнения | Точность     | Максимальное количество итераций | Комментарии (что было сделано)                                                                                                                                                                                                              |
|--------|------------------|--------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | 134.2 с          | 9.999302e-07 | 1000000                          | Базовый вариант: использовались две матрицы current/next). На каждом шаге происходило явное копирование данных из временного массива в основной через отдельный цикл. Это создавало колоссальное узкое место при постоянном обмене данными. |

|   |        |              |         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---|--------|--------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | 18.7 с | 9.999302e-07 | 1000000 | <p>Оптимизированный вариант,<br/>Применен директивный блок <code>#pragma acc data</code>, что позволило один раз скопировать сетку на устройство и забрать результат только в конце. Физическое копирование массивов заменено на <code>swar</code> указателей. Двумерные циклы раскрыты через <code>collapse(2)</code>, для поиска максимума невязки применен <code>reduction(max:err)</code>, а флаг <code>independent</code> снял ложные зависимости данных.</p> |
|---|--------|--------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Вывод:

В ходе выполнения лабораторной работы был написан и исследован C++-алгоритм решения уравнения теплопроводности методом простых итераций. Успешно реализованы граничные условия *via* линейной интерполяции, настроен парсинг аргументов через `boost::program_options` и сохранение результата в файл.

Анализ профилей и временных замеров показал, что «узким горлышком» данной задачи является не вычислительная мощность, а пропускная способность подсистемы памяти (Memory Bandwidth Bound). Отношение числа арифметических операций к количеству обращений к памяти крайне мало (на каждую итерацию требуется чтение 4-х соседних элементов и запись 1 нового).

Перенос на GPU и устранение лишних копирований между Host и Device (за счет использования *data-регионов* и обмена указателями вместо копирования массивов) позволил сократить время расчета более чем в 7 раз по сравнению с базовой версией на видеокарте. При этом даже оптимизированный GPU показывает ограниченную эффективность на малых сетках (128×128) из-за накладных расходов на инициализацию и передачу данных, однако на сетке 1024×1024 ускорение относительно многоядерного CPU становится весьма существенным.